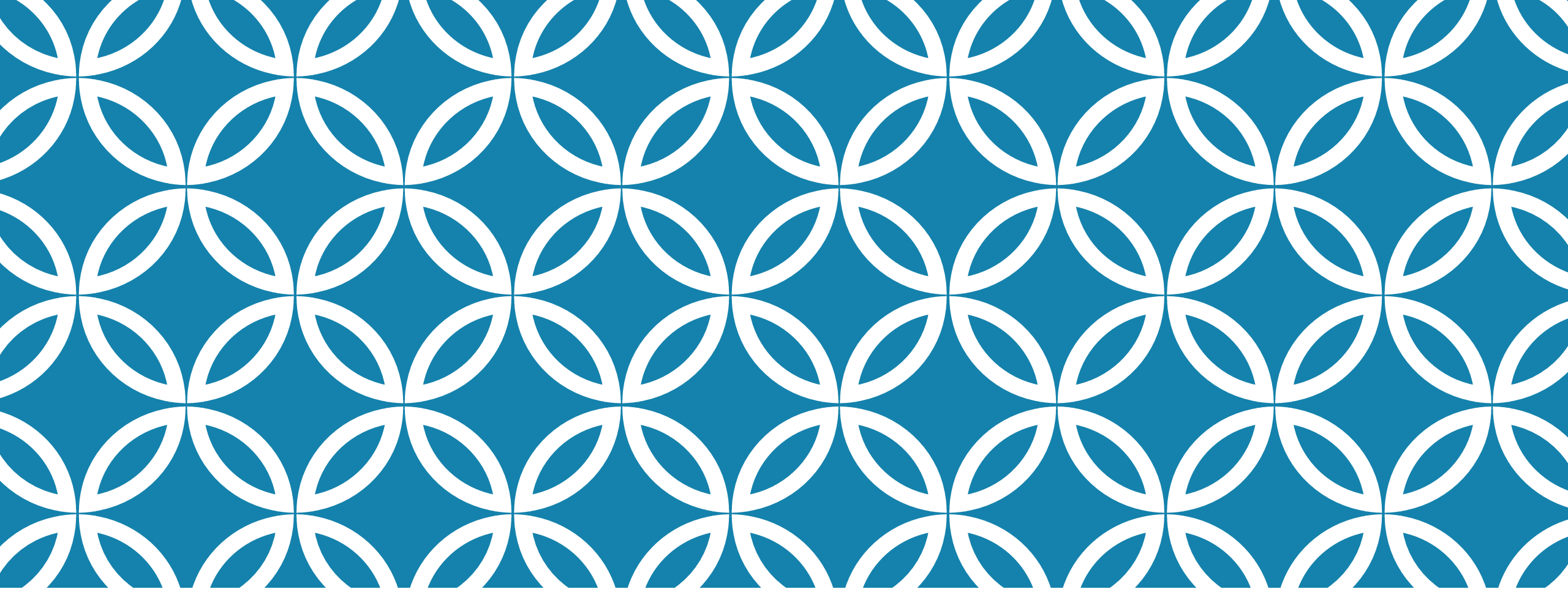




HTML 与 CSS 基础

从标签结构到移动端页面

目标：独立写出一个 375px 宽的移动端静态页

目录

一、HTML基础 —— 用什么标签搭结构

语义化标签 · 文本与列表 · 图像与链接 · 表单控件 · 全局属性

二、CSS基础 —— 用什么规则控制外观

① 选择器与盒模型 ② 布局 (Flex / vw·vh·% / rpx) ③ 工程化样式基础 (CSS 变量)

三、作业 —— 独立完成一份 375px 移动端静态页面

HTML 结构要求 7 条 · CSS 样式要求 7 条 · 交付与自查

要解决的 3 个问题

① 用什么标签搭结构？

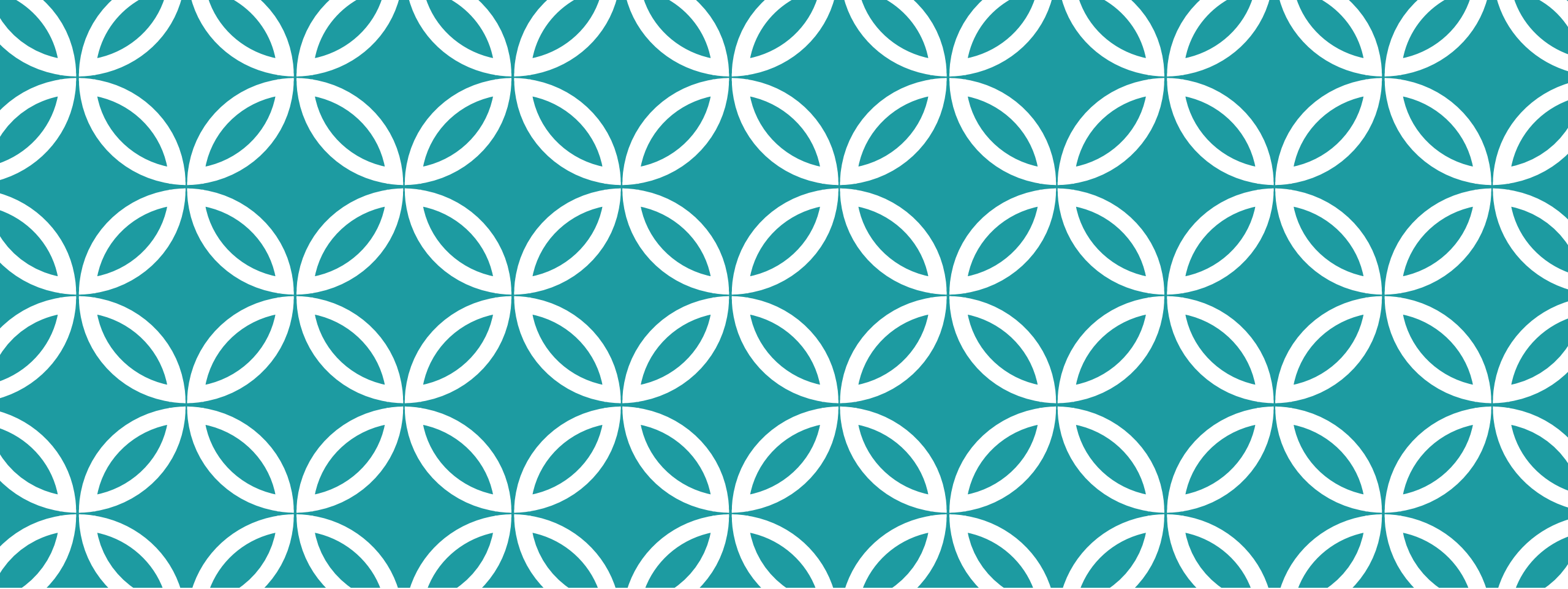
→ HTML 语义化标签：不是"通篇 div"，而是 header / nav / section / footer

② 用什么规则控制外观？

→ CSS 选择器选中元素 + 盒模型控制空间 + Flex 控制排列

③ 怎么让它适配手机屏？

▪ → viewport + border-box + Flex + vw / vh / % + rpx



一、HTML基础

学完你能写出：语义化骨架 +
列表 + 图文 + 完整表单

HTML 是什么

- HTML = Hyper Text Markup Language, 超文本标记语言
- 它不是编程语言, 是"标记语言": 用标签 (tag) 描述内容的结构与含义
- 一个标签 + 内容 = 一个元素 (Element); 标签推荐使用小写
- HTML 文本本质是纯文本, 由浏览器"翻译"成我们看到的网页
- 1990 年创立, 独立于操作系统平台, 靠浏览器识别与渲染

一句话: HTML 决定"页面上有什么"

浏览器开发者工具（必会）

- 打开方式：F12 / Ctrl+Shift+I / 右键 → 检查
- 移动模拟：Ctrl+Shift+M → 选 iPhone SE，尺寸 375 × 667
- **完成三件事：**
 - ① Elements 面板：看真实 DOM 结构，确认语义标签有没有用对
 - ② Computed / 盒模型图：看 padding、border、margin 实际数值
 - ③ 左右拖动页面：检查有没有出现横向滚动条

HTML 元素(Element)语法

- 元素 = 开始标签 + 内容 + 结束标签，标签推荐小写
- 属性写在开始标签中，名称/值对形式，属性值推荐加双引号
- 空元素没有内容，在开始标签中关闭：br / img / input / meta

```
<p>This is a paragraph</p>
```

```
<body bgcolor="yellow">
```

```
<table border="1">
```

```
<br />
```

```
<!-- 段落：开始标签 + 内容 + 结束标签 -->
```

```
<!-- 属性写在开始标签里 -->
```

```
<!-- 属性值用双引号 -->
```

```
<!-- 空元素：在开始标签中关闭 -->
```

```
<!-- 这是一段注释，不会在浏览器中显示 -->
```

```
<p>To break<br />lines<br />in a<br />paragraph,<br />use the br tag.</p>
```

文档结构与嵌套规则

■ 基本骨架：DOCTYPE → html → head + body

■ 嵌套三原则：

- ① 必须正确闭合 —— 有开始标签就要有结束标签
- ② 不能交叉嵌套 —— 后打开的必须先关闭
- ③ 块级元素可包含行内元素；行内元素内不要再放块级元素

```
<!-- 正确 -->  
<b><i>重点文字</i></b>
```

```
<!-- 错误：交叉嵌套 -->  
<b><i>重点文字</b></i>
```

```
<!-- 正确 -->  
<div><p>段落</p></div>
```

```
<!-- 错误：p 里放 div -->  
<p><div>区块</div></p>
```

```
<!DOCTYPE html>  
<html>  
  <head><title>页面标题</title></head>  
  <body><p>Hello</p></body>  
</html>
```


移动端第一步：viewport

- 必须写在 <head> 里，移动端页面的第一道门槛：

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

width=device-width：视口宽度 = 设备宽度（iPhone SE 为 375px）

initial-scale=1.0：初始缩放比例为 1 倍

不写的后果：手机浏览器按 980px 虚拟视口渲染，页面被整体缩小、字看不清

更糟的是所有 vw / % 单位全部算错，布局彻底失控

语义化布局标签

- header: 页头 / 顶部区域 (Logo、标题)
- nav: 导航链接区
- section: 文档中的一个有主题的区块
- footer: 页脚 / 底部区域 (版权、备案)
- div: 无语义的通用容器 —— 只在确实没有合适语义标签时才用

✗ 通篇 div (不推荐)

```
<div class="header">  
  <div class="logo">...</div>  
</div>  
<div class="nav">...</div>  
<div class="main">...</div>  
<div class="bottom">...</div>
```

✓ 语义化 (作业要求)

```
<header>  
  <div class="logo">...</div>  
</header>  
<nav>...菜单...</nav>  
<section>...商品列表...</section>  
<footer>...版权...</footer>
```

文本标签： p / h1–h6 / span

- <p> 段落：前后自动换行，段间距大于行间距
- <h1> ~ <h6> 六级标题，h1 最重要、h6 最次要
 - 选标题只看"结构层级"，不要为了把字变大而乱用 h 标签
- 行内容器：标记文本的一小段，不中断文本连续性
 - 与 <div> 的区别：div 是块级元素（独占一行），span 是行内元素

```
<h1>页面主标题</h1>
```

```
<h2>二级标题</h2>
```

```
<p>这是一个段落，段落前后会自动换行。</p>
```

```
<!-- 这位姑娘有一双蓝色的眼睛 -->
```

```
<p>这位姑娘有一双<span style="color:blue">蓝色</span>的眼睛。</p>
```

列表：ul / ol / li

- 无序列表 ：子项用 (list item)，默认圆点
- 有序列表 ：子项同样用 ，默认数字序号
- / 的直接子元素只能是 ，不能塞别的东西
 - 了解：自定义列表 <dl> → <dt> (项) + <dd> (项的描述)

```
<ul class="goods-list">                                <!-- 无序列表：商品列表 -->
  <li>商品 A</li>
  <li>商品 B</li>
</ul>

<ol>                                                    <!-- 有序列表：步骤 -->
  <li>加入购物车</li>
  <li>填写收货信息</li>
</ol>
```

图像

- 是空元素：只有属性，没有结束标签
- src: source, 图片地址 —— 必填
- alt: 图片无法显示时的替代文本 —— 必填（作业要求，漏写扣分）
- width / height: 宽高，可用数值（px）或百分比
- 作业的图片可以写网络地址，不引入本地文件

```

```

```

```

```

```

链接 <a> 与锚点

- <a> 用 href 属性指向目标；超文本的核心就是"能链出去"
- 外部链接：href 写完整网址，target="_blank" 新标签页打开
- 页面内锚点（两段配合，缺一不可）：
 - ① 在目标位置做标记：提示 或 <h2 id="cart">购物车</h2>
 - ② 在入口处写跳转：跳到提示 / 跳到购物车

```
<!-- 外部链接 -->
<a href="https://www.w3school.com.cn/" target="_blank">Visit W3School!</a>

<!-- 页面内锚点：标记 + 跳转 -->
<a href="#tips">跳到"有用的提示"</a>
...
<a name="tips">有用的提示</a>

<!-- 跨页锚点 -->
<a href="page.html#tips">另一页的提示</a>
```

表单 (1) : <form> 与 <input>

- <form> 是所有表单控件的容器，用于收集用户输入
- <input> 的形态由 type 决定：
 - text 单行文本 / password 密码 / number 数字 / email 邮箱 / tel 电话
- placeholder: 输入框里的灰色提示文字
- value: 输入框的默认值 (也是按钮上显示的文字)
- disabled: 禁用该控件，不可编辑、不会被提交
 - 提示: 未设置 name 的 input 将不会被提交

```
<input type="text"      placeholder="请输入用户名">
<input type="password" placeholder="请输入密码">
<input type="number"   value="1">
<input type="email"    placeholder="example@mail.com">
<input type="text"     value="不可修改" disabled>
```

表单（2）：textarea / select / button

- `<textarea>` 多行文本输入框，rows / cols 控制可见尺寸
- `<select>` 下拉选择框，子项用 `<option>`；可加 `selected` 设默认项
- `<button type="submit">` 提交按钮 —— 用它替代 `<input type="submit">`

```
<form action="/order" method="post" target="_self">
  <input type="text" name="name" placeholder="收货人姓名">
  <input type="tel" name="phone" placeholder="手机号">
  <input type="text" name="code" value="SH-2026" disabled>

  <textarea rows="3" placeholder="备注：如送货时间"></textarea>

  <select name="city">
    <option value="gz" selected>广州</option>
    <option value="bj">北京</option>
  </select>

  <button type="submit">提交订单</button>
</form>
```


表单 (3) : method —— get 与 post

- GET (HTML 表单的默认方式) : 把数据以"名称/值对"追加到 URL 后面
 - 适合非敏感数据; 结果可加书签; URL 长度限制 2048 字符
 - 例: `http://example.com/action_page.php?username=john&password=12345`
- POST: 数据放在 HTTP 请求正文中, 不在 URL 里显示
 - 没有大小限制, 可发送大量数据; 提交结果无法加书签
- 注意: POST 也不等于绝对安全, HTTP 是明文传输, 敏感数据要走 HTTPS

```
<form action="/action_page.php" method="post">    <!-- 推荐 -->
<form action="/search.php"      method="get">    <!-- 查询类 -->
```

表单（4）：target 属性

- target 决定“服务器返回的响应显示在哪里”：
- _self —— 当前窗口打开（默认值）
- _blank —— 新窗口 / 新标签页打开
- _parent —— 在父框架中打开
- _top —— 在整个窗口中打开，跳出所有框架

```
<form action="/order" method="post" target="_blank">
```

```
...
```

```
</form>
```

```
<!-- 同一个 target 规则也适用于 <a> 标签 -->
```

```
<a href="help.html" target="_blank">帮助中心</a>
```

核心全局属性速查表

属性	作用	常用元素	示例
id	唯一标识，页面中不可重复	所有（多用于锚点）	id="cart"
class	类名，可复用、可多个空格分隔	所有（首选写样式）	class="card hot"
src	外部资源地址	img	src="a.png"
href	链接目标地址	a、link	href="#cart"
value	元素的值	input、button、option	value="提交"
type	决定元素的类型	input、button	type="email"
placeholder	输入前的灰色提示	input、textarea	placeholder="手机号"
disabled	禁用控件（无值属性）	input、button、select	disabled

参考：多媒体标签 audio / video / object

- <audio> 音频标签：常配 controls 显示播放控件
- <video> 视频标签：可设 width、poster 封面图
- <object> 定义外部资源容器（网页、图片、播放器、插件）

```
<audio src="music.mp3" controls></audio>
```

```
<video src="intro.mp4" controls width="100%"></video>
```

```
<object data="demo.html" width="100%" height="300"></object>
```

HTML(不限于HTML)的学习网站

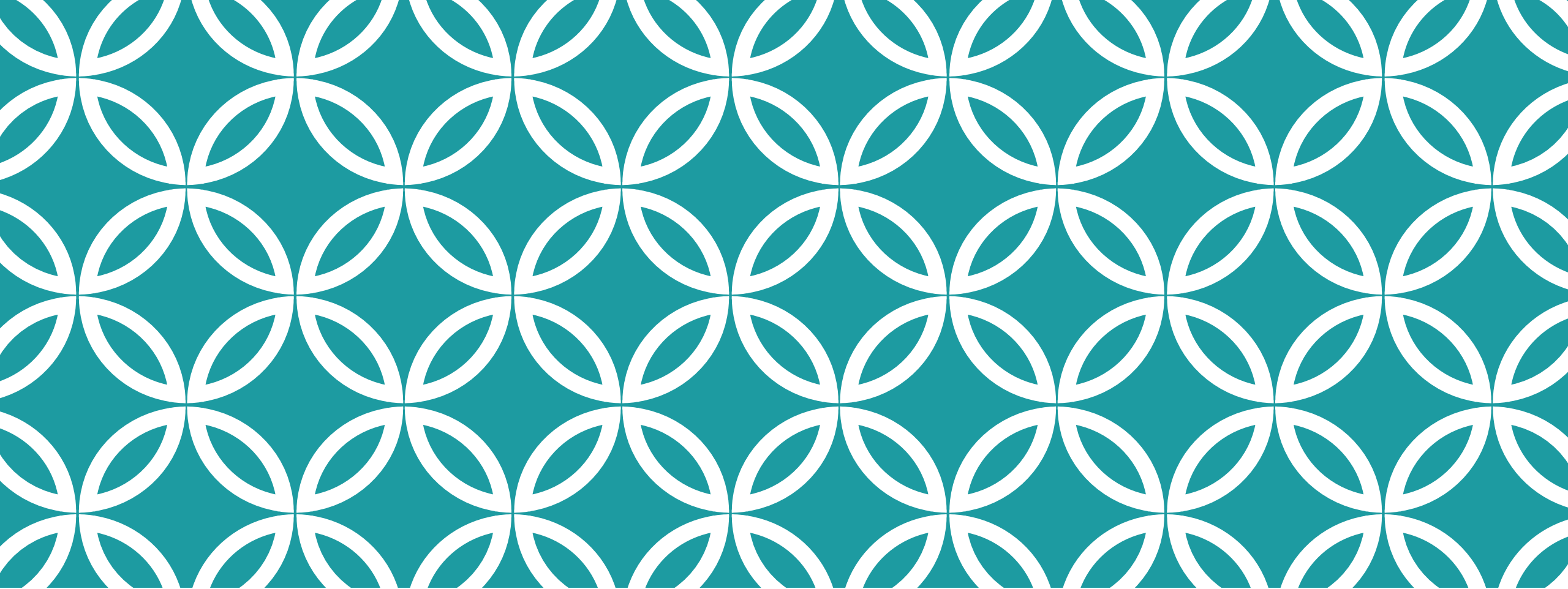
RUNOOB.COM



W3school

廖雪峰的官方网站





二、CSS基础

① 选择器与盒模型 ② 布局
(Flex / vw·vh·% / rpx) ③ 工程
化样式基础 (CSS 变量)

CSS 是什么 & 与 HTML 的关系

- CSS = Cascading Style Sheets, 层叠（级联）样式表
- 网页是多层结构, CSS 逐层设置样式, 最终看到的是最上面一层
- 分工: HTML 是网页的结构（有什么）, CSS 是网页的样式（长什么样）
- 组成不同: HTML 由标签组成, CSS 由选择器 + 声明块组成
- 注释不同: HTML 用 `<!-- -->`, CSS 用 `/* */`（可跨多行）
- HTML 文件可以包含 CSS（`<style>`）, 但 CSS 里不能写 HTML 标签

CSS 三种使用方式 + 语法

- 外部样式表：保存为 .css 文件，<link> 引入，多页复用
- 内部样式表：<style> 写在 <head> 中，单页使用
- 内联样式：style 作为元素属性，只作用于个别元素，优先级最高

<!-- 语法：选择器 + 声明块 -->

选择器 { 属性名：属性值； 属性名：属性值； }

```
.card {  
  width: 100%;           /* 注意：CSS 用冒号，HTML 属性用等号 */  
  padding: 12px;  
  color: #333;  
}
```


三大基础选择器

- 标签选择器 `p { }`: 选中页面中所有该标签
- 类选择器 `.card { }`: 选中所有 `class="card"` 的元素 —— 最常用
- ID 选择器 `#cart { }`: 选中唯一的那一个元素
- 取舍原则: 写样式优先用 `class`; `id` 留给锚点定位和 JS 操作
- 优先级: ID 选择器 > 类选择器 > 标签选择器

```
p      { color: #333; }      /* 标签选择器: 选中所有 <p> */
.card  { padding: 12px; }    /* 类选择器: 选中 class="card" */
#cart  { font-size: 16px; }  /* ID 选择器: 选中 id="cart" */

<p class="card" id="cart">...</p>
```

后代选择器 与 子代选择器

- 后代选择器（空格）：选中指定元素内部的所有该元素 —— 管"子"也管"孙"
- 子代选择器（>）：只选中指定元素的直接子元素 —— 管"子"不管"孙"

```
<div class="box">
  <p>直接子代</p>           <!-- 会被 div p 和 div>p 都选中 -->
  <section>
    <p>孙子代</p>           <!-- 只被 div p 选中 -->
  </section>
</div>
```

```
/*      后代选择器：空格      */
.box p { color: red; }
/* 命中：直接子代 + 孙子代 (2 个) */

/*      子代选择器：>      */
.box > p { color: red; }
/* 命中：只有直接子代 (1 个) */
```

CSS 盒模型

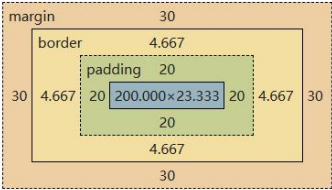
- CSS 把每个元素都看成一个盒子，从内到外共四层：
 - ① content 内容区 (width / height)
 - ② padding 内边距 (内容与边框之间)
 - ③ border 边框
 - ④ margin 外边距 (盒子与盒子之间)
- 元素实际占用宽度 = content + padding + border + margin
- 用 F12 的盒模型图可以直观看到每一层的数值



```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS盒模型演示</title>
  <style>
    .box {
      width: 200px;      /* 内容宽度 */
      padding: 20px;     /* 内边距 */
      border: 5px solid #e74c3c; /* 边框 */
      margin: 30px;      /* 外边距 */
      background-color: #ecf0f1; /* 背景色，方便看到内容区 */
    }
  </style>
</head>
<body>
  <div class="box">
    内容区宽度200px
  </div>
  <!-- 加一个参照元素，展示margin的影响 -->
  <div style="background-color: #3498db; color: white; padding: 5px;">
    下方的蓝色块用来观察上面元素的margin
  </div>
</body>
</html>
```



下方的蓝色块用来观察上面元素的margin



padding / border / margin 的简写取值

- 单一值：四个方向相同（上、右、下、左）
- 两个值：第一个管上下，第二个管左右
- 三个值：第一个管上，第二个管左右，第三个管下
- 四个值：按 上 → 右 → 下 → 左 顺时针依次分配

```
padding: 20px;                /* 上下左右全 20px */
padding: 10px 20px;           /* 上下 10px, 左右 20px */
padding: 10px 20px 30px;      /* 上 10, 左右 20, 下 30 */
padding: 10px 20px 30px 40px; /* 上 10, 右 20, 下 30, 左 40 */

/* border 与 margin 的简写规则完全相同 */
```

box-sizing: border-box

- 默认 content-box: width 只算内容区, 加 padding / border 会把盒子"撑大"
- border-box: width 包含 padding 和 border, 盒子总宽固定不变

content-box (默认)

```
.card {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid #333;  
}  
/* 实际占用宽度  
   = 200 + 20*2 + 5*2  
   = 250px ← 被撑大了 */
```

border-box (推荐)

```
.card {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid #333;  
}  
/* 实际占用宽度 = 200px  
   内容区被压缩到 150px */
```

常用样式属性（上）：文字排版

- font-size: 14px —— 字号。移动端正文常用 14~16px
- font-weight: 400 / 700 / bold —— 字重，400 常规、700 加粗
- color: #333333 —— 文字颜色，可用十六进制 / rgb() / 颜色名
- text-align: left / center / right —— 文本水平对齐
- line-height: 1.5 —— 行高，写倍数最省心（随字号自适应）

```
.title {  
  font-size: 18px;  
  font-weight: 700;  
  color: #333333;  
  text-align: center;  
  line-height: 1.5;  
}
```

常用样式属性（下）：背景 / 尺寸 / 圆角 / 阴影

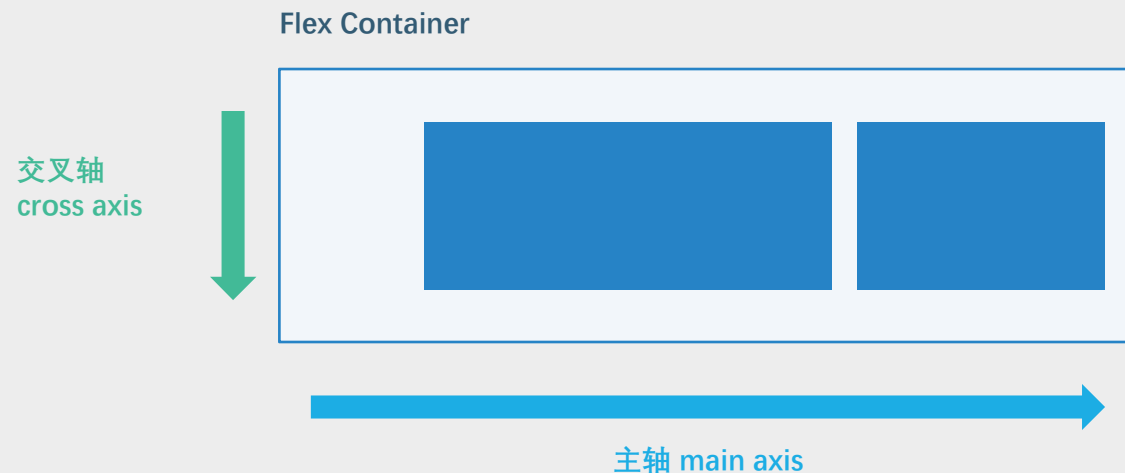
- background / background-color: 背景（色），也可写渐变或图片
- width / height: 宽度与高度，可用 px、%、vw
- border-radius: 圆角。写 50% 可把正方形变成圆形（头像常用）
- box-shadow: 阴影，格式 X偏移 Y偏移 模糊 颜色

```
.card {  
  width: 100%;  
  height: 120px;  
  background-color: #ffffff;  
  border-radius: 12px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);  
}
```

```
.avatar {  
  width: 40px; height: 40px;  
  border-radius: 50%;      /* 变圆形 */  
}
```


Flexbox（弹性盒子）基本概念

- 一维布局模型：做水平或垂直方向上的排列、对齐和空间分配
- 弹性容器 Flex Container：设置了 `display: flex` 的元素
- 弹性项目 Flex Item：容器的直接子元素，自动成为项目
- 主轴 Main Axis：项目排列的方向，由 `flex-direction` 决定
- 交叉轴 Cross Axis：垂直于主轴的方向
- `flex-flow`：`flex-direction` 与 `flex-wrap` 的简写



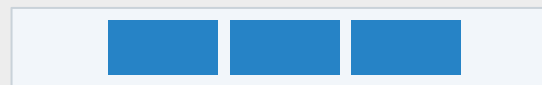
Flex 容器属性（写在容器上）

- `display: flex` —— 开启弹性布局（`inline-flex` 为行内容器）
- `flex-direction: row`（默认横排） / `column`（竖排）
- `justify-content`：主轴方向对齐 —— 见右侧五种取值
- `align-items`：交叉轴对齐 —— `stretch` / `flex-start` / `center` / `flex-end`
- `flex-wrap: nowrap`（默认不换行） / `wrap`（空间不足时换行）

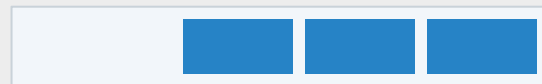
`flex-start`



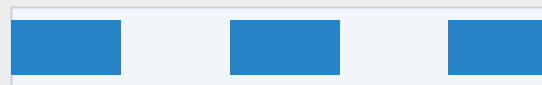
`center`



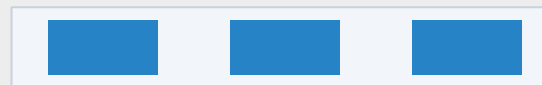
`flex-end`



`space-between`



`space-around`



Flex 项目属性（写在项目上）

- flex: 1 —— grow / shrink / basis 的简写，等分容器剩余空间
 - 底部导航栏三等分、输入框与按钮并排，都靠它
- flex-grow: 1 —— 有剩余空间时的放大比例（默认 0 不放大）
- flex-shrink: 0 —— 空间不足时不缩小（配合固定宽度使用）
- align-self: center —— 单个项目覆盖容器的 align-items

```
.toolbar { display: flex; }           /* 容器 */
.toolbar > div { flex: 1; }          /* 三个项目等分宽度 */

.search { flex: 1; }                 /* 输入框占满剩余空间 */
.btn     { flex-shrink: 0; width: 72px; } /* 按钮固定 72px 不被压缩 */

.special { align-self: center; }      /* 只有它垂直居中 */
```

移动端适配单位：vw / vh / %

- vw: $1\text{vw} = \text{视口宽度的 } 1\%$ → 375px 屏下 $1\text{vw} = 3.75\text{px}$, $100\text{vw} = 375\text{px}$
- vh: $1\text{vh} = \text{视口高度的 } 1\%$ → 667px 屏下 $100\text{vh} = 667\text{px}$
- %: 相对父元素的尺寸, 不是相对视口 —— 两者别搞混
- 用法: 满宽横幅用 100vw ; 首屏高度用 100vh ; 卡片宽度用 % 随父容器
- 注意: 部分浏览器 100vw 含滚动条宽度, 满宽布局优先用 100%

单位	含义	375×667 屏实际值
100vw	视口宽度 100%	375px
50vw	视口宽度一半	187.5px
100vh	视口高度 100%	667px
1vw	视口宽度 1% #	3.75px

rpx 与 UniApp 多端适配

- rpx (responsive pixel) 是跨端框架使用的相对单位
- 基准：任何屏幕的宽度都等于 750rpx
- 换算公式： $1\text{rpx} = \text{屏幕宽度} / 750$
 - iPhone SE (375px) : $1\text{rpx} = 0.5\text{px}$, $750\text{rpx} = 375\text{px}$
 - iPhone XR (414px) : $1\text{rpx} = 0.552\text{px}$, $750\text{rpx} = 414\text{px}$
- 实现原理：编译期把 rpx 换算为 rem，运行时由 JS 按屏幕实际宽度
 - 动态设置根元素 font-size，从而实现整页等比例缩放
- 与 vw 的关系： $100\text{vw} = 750\text{rpx}$ ，即 $1\text{rpx} = 0.1333\text{vw}$

练习：设计稿 rpx → 实际 px 换算

设计稿宽度 750rpx，请把下列标注换算到两台设备（ $px = rpx \times \text{屏幕宽} \div 750$ ）

元素	设计稿标注	375px 屏	414px 屏
顶部导航高度	88rpx	44 px	48.6 px
卡片宽（两张并排）	340rpx	170 px	187.7 px
按钮宽度	686rpx	343 px	378.7 px
元素间距	24rpx	12 px	13.2 px

思考：为什么按 750rpx 出设计稿，同一份样式能在大屏小屏都保持同样比例？

（750rpx 恒等于屏幕宽度，框架运行时动态换算，故视觉比例不变）

CSS 变量（自定义属性）：定义与引用

- 定义：以 `--` 开头，通常写在 `:root` 里表示全文档可用
- 引用：用 `var( 变量名 )` 取值
- 作用域：定义在哪个选择器上，就在它及其后代生效；会向下继承

```
/* 1. 定义：写在 :root 表示全局 */
:root {
  --primary-color: #ff6b35;
  --border-radius: 12px;
  --font-size-base: 14px;
}

/* 2. 引用：用 var() */
.btn {
  background: var(--primary-color);
  border-radius: var(--border-radius);
  font-size: var(--font-size-base);
}
```

CSS 变量与主题切换

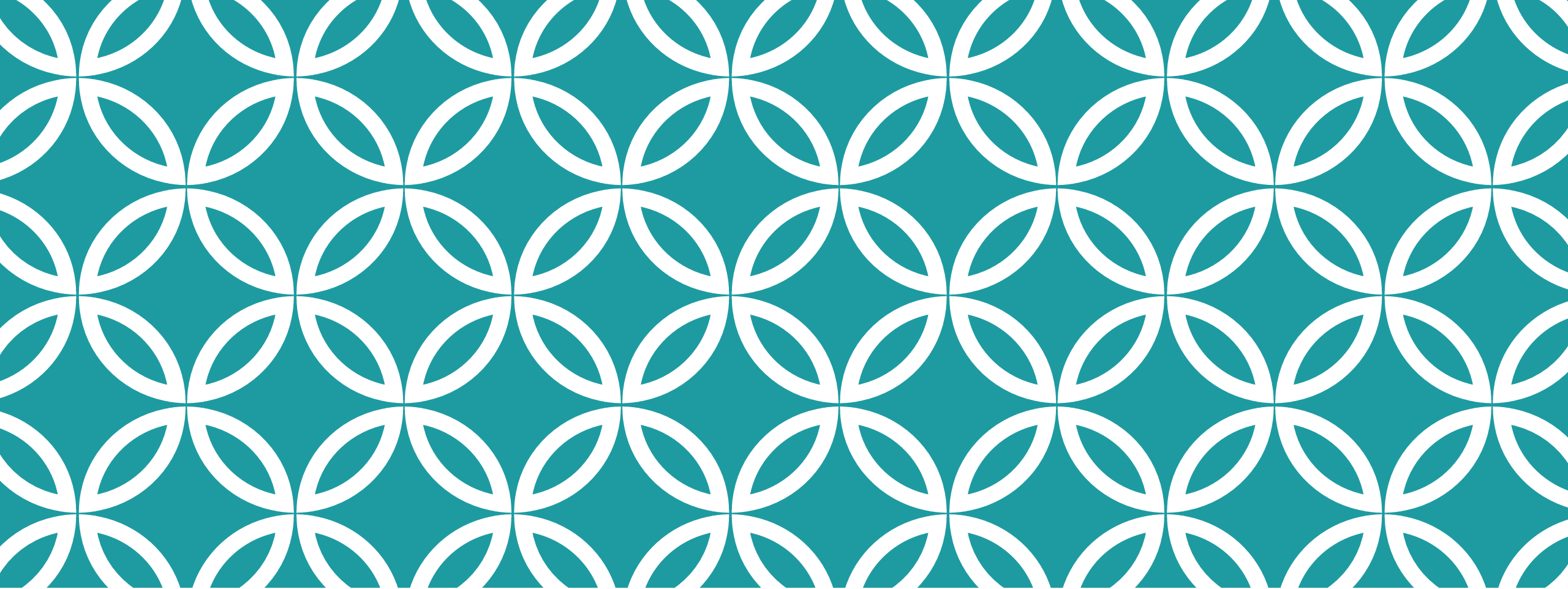
- 换主题 = 只改 :root 里的变量值，全站同步生效
- 改 1 处（变量） vs 改 30 处（每个选择器里硬编码颜色）—— 设计系统的做法

```
:root { --primary-color: #ff6b35; --bg-color: #ffffff; }
```

```
/* 深色主题：只覆盖变量，业务样式一行都不用改 */
```

```
.theme-dark {  
  --primary-color: #ff8c5a;  
  --bg-color:      #1f2328;  
}
```

```
.card { background: var(--bg-color); }  
.btn   { background: var(--primary-color); }
```

三、作业

独立完成一份 375px 宽的移动端静态页面

作业任务与页面分区示意

- 任务：独立完成一份移动端静态页面，宽度模拟 375px 视口
- 页面至少包含三个区域：
 - ① 顶部导航栏：Logo / 导航链接 / 用户图标（Flex 横向分布）
 - ② 中间内容区：ul 商品列表（ ≥ 3 张图）或横向滚动卡片
 - ③ 底部表单区：输入框与按钮水平对齐（Flex）
- 附一份 200字以内的说明文档：页面主题 + 各区域的业务功能



交付要求（一）：HTML 结构 · 共 7 条

1. 用 header / nav / section / footer 搭骨架，不能通篇 div
2. 用 ul 或 ol 呈现一组列表数据（商品、菜单、目录均可）
3. 至少 3 张图片，src 与 alt 两个属性都必须写
4. 至少 2 处 a 标签（页面内锚点跳转或外部链接）
5. form 内包含：≥2 种 input + 1 个 textarea + 1 个 select + 1 个 button
6. form 必须显式写出 method 与 target
7. 正确使用 id / class / placeholder / value / type / disabled

交付要求（二）：CSS 样式 · 共 7 条

1. 主要样式用类选择器；ID 选择器只用于唯一的锚点定位元素
2. 至少 1 处后代选择器或子代选择器，用于限定特定容器内的元素
3. 正确应用盒模型：至少包含 padding、margin、border 的设置
4. 使用 box-sizing: border-box 全局重置盒模型计算方式
5. 用 Flex 完成三个区域的排版（顶部导航 / 中间内容 / 底部表单）
6. 定义并使用至少 3 个 CSS 变量，并在全局样式中引用
7. 使用 vw 或 % 单位实现至少 1 处元素的宽度自适应

交付要求（三）与自查表

■ 交付物：

- ① 一份完整的 .html 文件，所有 CSS 内嵌于 <style>，不引入外部文件
- ② 在浏览器移动设备模拟（375 × 667）中打开：完整显示、无横向滚动条
- ③ 说明文档：页面主题 + 各区域业务功能

■ 自查 6 步：

- 1. F12 → 切到 375×667 移动模拟 2. 左右拖动，确认无横向滚动条
- 3. 每张 img 都写了 alt 4. :root 里 ≥3 个变量且被真正引用
- 5. 点锚点能跳转；form 上同时有 method 和 target
- 6. 全局已重置 * { box-sizing: border-box; }

常见翻车点 Top 5

- ① 忘写 viewport → 页面被整体缩小、vw 全部算错，布局失控
 - 解决：<head> 里加 <meta name="viewport" content="width=device-width, initial-scale=1.0">
- ② width: 100% 又加 padding → 元素溢出父容器
 - 解决：全局 * { box-sizing: border-box; }
- ③ 出现横向滚动条 → 检查固定 px 宽度、100vw、负 margin
 - 解决：宽度改用 % / vw，或给父容器加 overflow-x: hidden
- ④ 拿 id 写样式 → 违背了"id 只用于锚点定位"的要求
 - 解决：样式一律改用 class
- ⑤ 图片没写 alt → 直接不符合作业第 3 条